

Role 2: Backend Development

Domain: Server Architecture, REST API Engineering, Streaming Protocols & Middleware

Core Technology: Node.js (ES Modules) • Express 5 • Pino HTTP • SSE Streaming • CORS

1. Executive Summary & Scope

The **Backend Developer** designs, builds, and maintains the server-side infrastructure of SatyaCheck. This role functions as the central operational bridge connecting the client-side user interface (Role 1), the relational/file persistence database layer (Role 3), and the artificial intelligence verification runtime (Role 4).

Core Engineering Objectives:

- Architect a pure JavaScript, native ES Module HTTP server on Express 5 without compilation overhead.
- Enforce strict request boundary limits (50MB body limit for base64 image transfers, CORS whitelist).
- Build a real-time Server-Sent Events (SSE) streaming endpoint (/api/chat) for token-by-token AI answers.
- Implement defensive error handling so third-party failures or database dropouts never result in unhandled 500 crashes.
- Provide high-throughput structured JSON logging using pino and pino-http.

2. Complete Technology Stack

Technology	Version	Role & Architectural Rationale
Node.js	v20+ (LTS)	High-performance asynchronous JavaScript runtime running native ECMAScript Modules (import/export).
Express	5.0.1	Modernized web framework with native Promise error handling and enhanced route matching.
Pino & Pino-HTTP	9.6.0 / 10.4.0	Extreme-performance JSON structured logger; operates up to 5x faster than standard Winston or Morgan.
CORS Middleware	2.8.5	Manages Cross-Origin Resource Sharing headers for local frontend proxying and production origins.
OpenAI SDK	7.10.0	Used in Ollama-compatible mode (http://127.0.0.1:11434/v1) with zero cloud vendor lock-in.

3. Comprehensive REST & Streaming API Specification

HTTP Method	Route	Input Payload / Parameters	Output Response & Status
GET	/api/healthz	None	{ status: "ok" } (200 OK)
POST	/api/analyze	{ content: string, type: "text" "url" "image" }	Full 10-dimension forensic dossier (200 OK)
POST	/api/chat	{ messages: [{ role: string, content: string }] }	SSE Stream (text/event-stream) yielding tokens
POST	/api/credibility	{ source: string } (Domain or publication)	{ name, credibilityScore, bias, type, transparency }
GET	/api/stats	None	{ total, realCount, fakeCount, trendingTopics: [] }
GET	/api/history	None	Array of last 20 verified claims with timestamps
GET	/api/trending	None	Curated list of 5 circulating misinformation stories
GET	/api/quiz	None	10-question misinformation detection challenge

4. Core Code Implementations

4.1 Server Bootstrapper & Error Lifecycle (src/index.js)

```
import app from "./app.js";
import { logger } from "./lib/logger.js";

const port = Number(process.env.PORT || 3000);

if (Number.isNaN(port) || port <= 0) {
  throw new Error(`Invalid PORT value: "${process.env.PORT}"`);
}

const server = app.listen(port, (err) => {
  if (err) {
    logger.error({ err }, "Fatal error listening on port");
    process.exit(1);
  }
  logger.info({ port }, "SatyaCheck Backend Server listening successfully");
});

// Graceful process termination
process.on("SIGTERM", () => {
  logger.info("SIGTERM signal received. Closing HTTP server gracefully.");
  server.close(() => process.exit(0));
});
```

4.2 Application Configuration & Middleware (src/app.js)

```
import express from "express";
import cors from "cors";
import pinoHttp from "pino-http";
import path from "path";
import { fileURLToPath } from "url";
import router from "../routes/index.js";
import { logger } from "../lib/logger.js";

const app = express();

// High-speed structured logging
app.use(pinoHttp({
  logger,
  serializers: {
    req: (req) => ({ id: req.id, method: req.method, url: req.url?.split("?")[0] }),
    res: (res) => ({ statusCode: res.statusCode })
  }
}));

app.use(cors());
app.use(express.json({ limit: "50mb" })); // Accommodates high-res image uploads
app.use(express.urlencoded({ extended: true, limit: "50mb" }));

// Mount all API endpoints under /api
app.use("/api", router);

export default app;
```

4.3 Real-Time Server-Sent Events (SSE) Streaming (src/routes/chat.js)

```
router.post("/chat", async (req, res) => {
  const { messages: rawMessages } = req.body;
  if (!rawMessages || !Array.isArray(rawMessages)) {
    return res.status(400).json({ message: "messages array is required" });
  }

  // Set HTTP headers for persistent Server-Sent Events stream
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");
  res.setHeader("Access-Control-Allow-Origin", "*");

  try {
    const stream = await ollama.chat.completions.create({
      model: process.env.OLLAMA_MODEL || "llama3.2:1b",
      messages: rawMessages,
      stream: true,
    });

    for await (const chunk of stream) {
      const token = chunk.choices[0]?.delta?.content || "";
      if (token) {
        res.write(`data: ${JSON.stringify({ type: "content", content: token })}\n\n`);
      }
    }
    res.write(`data: ${JSON.stringify({ type: "done" })}\n\n`);
    res.end();
  } catch (err) {
    req.log?.error({ err }, "SSE streaming failed");
    res.write(`data: ${JSON.stringify({ type: "error", error: "Streaming failed" })}\n\n`);
    res.end();
  }
});
```

5. Architectural Resilience & Fault Tolerance

- **Database Decoupling:** The backend never allows database network timeouts to crash an analysis. If PostgreSQL is unreachable, analysis results are immediately captured in a local append-only JSON file (`analyses-storage.json`).
- **Memory & Timeout Caps:** Web scraping requests (via `AbortController`) time out after strictly 10 seconds to prevent denial-of-service hanging from slow remote target websites.
- **Payload Sanitization:** Request bodies are strictly validated before being passed to AI prompts, mitigating prompt injection vulnerabilities.

6. Viva & Technical Defense Q&A

Q1: Why did you choose Server-Sent Events (SSE) instead of WebSockets for the AI chat?

A: The AI assistant is unidirectional (the client sends a message once, and the server streams tokens back). SSE runs over standard HTTP/1.1 without complex WebSocket handshakes, supports automatic browser reconnection natively, and bypasses corporate proxy firewalls seamlessly.

Q2: Why did you remove TypeScript from the backend and use native ES Modules?

A: Pure Node.js ES Modules run natively with `node src/index.js`. This eliminates the build step (no `esbuild`, `tsc`, or `watcher` required), reduces memory consumption, and speeds up local developer reboot cycles to under 500ms.

Q3: How does the backend prevent Out-Of-Memory (OOM) errors during image verification?

A: Express body parsers enforce a strict 50mb cap, and images are processed as Base64 payloads directly passed to the vision pipeline without being stored as temporary clutter on disk.